

Structures de Données et Algorithmes Avancés

Lab S02 — Analyse algorithmique

Compter, mesurer, confronter

Master 1 — 2025–2026

Dr. El Hadji Bassirou TOURÉ
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Avant de commencer

Prérequis : TP-1 complété, CM S02 lu, Lab S01 terminé.

Organisation : deux parties. *Partie I* — vous instrumentez du code pour compter les opérations réellement exécutées, puis vous mesurez des temps d'exécution. *Partie II* — vous établissez par le calcul les formules que vos mesures auront confirmées.

Ce Lab n'est pas noté.

Livrable : le dépôt Git du projet `lab-s02` et le rapport individuel décrit en fin de document.

Ce que vous saurez faire à la fin de ce Lab

1. Instrumenter un fragment de code pour compter ses opérations élémentaires selon la convention du cours.
2. Vérifier expérimentalement une formule de comptage, et localiser l'écart quand elle est fausse.
3. Mesurer un temps d'exécution avec `System.nanoTime()` en évitant les pièges de mesure les plus courants.
4. Confronter une courbe mesurée à une prédiction asymptotique, et interpréter les écarts.
5. Prouver une borne $\mathcal{O}$ en exhibant c et n_0 .
6. Résoudre une récurrence de diviser-pour-régner.

Table des matières

I	Pratique : instrumentation et mesure	2
1	Environnement de travail	2
2	Étape 1 — Compter pour de vrai	2

3	Étape 2 — Un test qui force la formule	3
4	Étape 3 — Mesurer un temps d'exécution	4
5	Étape 4 — Observer	5
6	Enregistrer le travail	6
II Théorie : exercices d'application		7
7	Exercice 1 — Établir $f(n)$	7
8	Exercice 2 — Prouver une borne	7
9	Exercice 3 — Vrai ou faux	8
10	Exercice 4 — Analyse de cas	8
11	Exercice 5 — Récurrences	8
12	Exercice 6 — Question subsidiaire	9
	Pour aller plus loin (optionnel)	10
	Aide-mémoire	10

Première partie

Pratique : instrumentation et mesure

1 Environnement de travail

Listing 1 – Rappel : lancer le conteneur et créer le projet

```
# Windows (Git Bash)
MSYS_NO_PATHCONV=1 docker run -it -v "$(pwd):/workspace"
    elbachirtoure/dsa-java-m1:v1.1
# macOS / Linux
docker run -it -v "$(pwd):/workspace" elbachirtoure/dsa-java-m1:v1.1

cd /workspace
new-project lab-s02
cd lab-s02
gradle build --no-daemon
```

Deux règles héritées du TP-1

La classe principale s'appelle `Main`, et toutes les commandes Gradle prennent `-no-daemon`.

2 Étape 1 — Compter pour de vrai

Ce qu'on matérialise du CM

Le CM établit $f(n) = 5n + 3$ pour la somme des n premiers entiers, avec la convention : $s = s + i$ vaut deux opérations, $i++$ vaut deux opérations, et il y a $n + 1$ comparaisons. Un compteur incrémenté à chaque opération transforme cette formule en fait vérifiable.

Créez `src/main/java/Compteur.java` :

Listing 2 – `Compteur.java`

```
1 public class Compteur {
2     private long operations = 0;
3
4     public void ajouter(int k) { operations += k; }
5     public long total()      { return operations; }
6     public void remise()     { operations = 0; }
7 }
```

Puis `src/main/java/Fragments.java` : la boucle du CM, réécrite en déclarant chaque opération comptée.

Listing 3 – `Fragments.java` — somme instrumentée

```
1 public class Fragments {
2
3     /** Somme des n premiers entiers, instrumentée. */
4     public static long sommeInstrumentee(int n, Compteur c) {
5         int s = 0;                c.ajouter(1);    // affectation
6         int i = 0;                c.ajouter(1);    // affectation
7         while (true) {
```

```

8         c.ajouter(1);                                // comparaison i < n
9         if (!(i < n)) break;
10        s = s + i;                c.ajouter(2);        // addition +
11        affectation                c.ajouter(2);        // addition +
12        i = i + 1;                affectation
13    }
14    return s;
15 }

```

Pourquoi une boucle while et non un for

La forme `for` cache l'ordre exact des opérations : on ne voit pas que la comparaison est évaluée une fois de plus que le corps. En dépliant la boucle, chaque opération devient une ligne, et le compteur devient lisible. C'est un artifice pédagogique, pas un style de production.

Vérification 1 — à compléter

Ajoutez à `Fragments` une méthode `triangleInstrumente(int n, Compteur c)` correspondant au fragment du CM :

```
for (i = 0; i < n; i++) { for (j = i; j < n; j++) { c = c + 1; } }
```

N'oubliez pas de compter l'affectation `j = i` à chaque tour externe, ni la comparaison qui fait sortir de chaque boucle.

3 Étape 2 — Un test qui force la formule

Ce qu'on matérialise du CM

Une formule de comptage est une affirmation vérifiable. Un test `JUnit` qui compare le compteur à $5n + 3$ pour plusieurs valeurs de n transforme l'affirmation en propriété testée — et fait échouer le test si l'on a oublié une opération.

Listing 4 – `src/test/java/FragmentsTest.java`

```

1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class FragmentsTest {
5
6     @Test
7     void sommeSuitLaFormule() {
8         for (int n = 0; n <= 20; n++) {
9             Compteur c = new Compteur();
10            Fragments.sommeInstrumentee(n, c);
11            assertEquals(5L * n + 3, c.total(), "ecart a n = " + n);
12        }
13    }
14
15    @Test
16    void sommeCalculeLaBonneValeur() {

```

```

17     Compteur c = new Compteur();
18     assertEquals(45, Fragments.sommeInstrumentee(10, c)); //
        0+1+...+9
19 }
20 }

```

```
gradle test --no-daemon
```

Vérification 2 — à compléter

1. Supprimez le `c.ajouter(1)` de la comparaison, relancez les tests. Quel message obtenez-vous, et pour quelle valeur de n le test échoue-t-il en premier?
 2. Rétablissez-le, puis écrivez le test correspondant à `triangleInstrumente` : la formule attendue est $f(n) = 2,5n^2 + 7,5n + 3$. Comment l'écrire en arithmétique entière, sans nombres à virgule ?
- Aide : $2,5n^2 + 7,5n + 3 = (5n^2 + 15n + 6)/2$, et le numérateur est toujours pair.*

4 Étape 3 — Mesurer un temps d'exécution

Ce qu'on matérialise du CM

Le comptage donne $f(n)$; le chronomètre donne un temps. Les deux doivent raconter la même histoire : si f est quadratique, doubler n doit multiplier le temps par environ quatre. Ce rapport est l'observable qui relie la théorie à la machine.

Listing 5 – Benchmark.java — insertions en tête

```

1  import java.util.ArrayList;
2  import java.util.LinkedList;
3  import java.util.List;
4
5  public class Benchmark {
6
7      public static long mesurerInsertionsEnTete(List<Integer> liste,
8          int n) {
9          long debut = System.nanoTime();
10         for (int i = 0; i < n; i++) {
11             liste.add(0, i);
12         }
13         long fin = System.nanoTime();
14         return (fin - debut) / 1_000_000; // millisecondes
15     }
16
17     public static void main(String[] args) {
18         int[] tailles = {1_000, 2_000, 4_000, 8_000, 16_000, 32_000};
19
20         // Chauffe : la JVM compile le code apres quelques milliers
21         // d'executions.
22         for (int r = 0; r < 3; r++) mesurerInsertionsEnTete(new
23             ArrayList<>(), 2_000);
24     }
25 }

```

```

22      System.out.printf("%8s %14s %14s %10s%n", "n",
23          "ArrayList(ms)", "LinkedList(ms)", "rapport");
24      long precedent = 0;
25      for (int n : tailles) {
26          long tArray = mesurerInsertionsEnTete(new ArrayList<>(),
27              n);
28          long tLinked = mesurerInsertionsEnTete(new
29              LinkedList<>(), n);
27          String rapport = (precedent == 0) ? "-" :
28              String.format("%.1f", (double) tArray / precedent);
29          System.out.printf("%8d %14d %14d %10s%n", n, tArray,
30              tLinked, rapport);
31          precedent = tArray;
32      }
33  }
34  }

```

```
gradle run --no-daemon
```

Trois pièges de la mesure

La chauffe : les premières exécutions sont interprétées, les suivantes compilées. Sans boucle de chauffe, la première ligne du tableau est systématiquement fausse.

La granularité : en dessous de quelques millisecondes, le bruit domine. Si une ligne affiche 0, augmentez n .

L'élimination de code mort : si le résultat d'un calcul n'est jamais utilisé, la JVM peut supprimer le calcul. Ici, la liste est bien construite, donc le travail a lieu.

5 Étape 4 — Observer

Ce qu'on matérialise du CM

Le CM affirme que n insertions en tête coûtent $n(n-1)/2$ déplacements dans un tableau contigu et n rebranchements dans une liste chaînée. Ce programme affiche le rapport entre deux tailles consécutives : un rapport proche de 4 quand on double n signe un coût quadratique ; un rapport proche de 2, un coût linéaire.

Vérification 3 — notez vos observations

Remplissez avec vos propres mesures.

n	ArrayList (ms)	LinkedList (ms)	rapport ArrayList
1 000			—
2 000			
4 000			
8 000			
16 000			
32 000			

1. Quand n double, par combien le temps de l'ArrayList est-il multiplié? _____ Cela correspond-il à la prédiction du CM? _____
2. Et pour la LinkedList? _____
3. À partir de quelle taille l'écart devient-il visible à l'œil nu? _____
4. Vos rapports sont-ils exactement égaux à la prédiction? Citez une cause possible d'écart. _____

Vérification 4 — une seconde mesure

Écrivez une méthode `mesurerAccesParRang(List<Integer> liste, int n)` qui, sur une liste déjà remplie de n éléments, exécute n appels à `liste.get(i)` avec i tiré au hasard, et mesure le temps total.

Prédisez le résultat *avant* d'exécuter :

ArrayList : _____ LinkedList : _____

Puis exécutez. Votre prédiction était-elle correcte? _____

6 Enregistrer le travail

Listing 6 – Git

```
cd /workspace/lab-s02
git add .
git commit -m "Lab S02 : instrumentation, tests de comptage,
benchmarks"
```

Deuxième partie

Théorie : exercices d'application

Comment aborder cette partie

Ces exercices ne sont pas un contrôle et ne sont pas notés. Ils portent sur les fragments que vous venez d'instrumenter : le but est de retrouver par le calcul les nombres que votre compteur a affichés. Sauf mention contraire, on applique la convention du cours : $s = s + i$ compte pour deux opérations, $i++$ pour deux, une comparaison pour une, un accès `tab[i]` pour un.

7 Exercice 1 — Établir $f(n)$

Pour chaque fragment, donnez le nombre exact d'opérations puis la classe Θ .

(a)

```
int s = 0; for (int i = 0; i < n; i++) { s = s + tab[i]; }
```

Détail du décompte : _____
 $f(n) =$ _____ classe : _____

(b)

```
int c = 0; for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { c = c + 1; } }
```

$f(n) =$ _____ classe : _____

(c)

```
int t = 1; for (int i = 1; i < n; i = i * 3) { t = t + i; }
```

Nombre de tours de boucle : _____ classe : _____

Pour $n = 100$, listez les valeurs prises par i : _____

(d) Comparez (b) et le fragment triangulaire du CM ($j = i$ au lieu de $j = 0$). Les deux ont-ils la même classe? Le même $f(n)$? _____

8 Exercice 2 — Prouver une borne

(a) Montrez que $f(n) = 4n + 25 \in \mathcal{O}(n)$ en exhibant un couple (c, n_0) et en rédigeant l'inégalité.

(b) Le CM démontre $3n^2 + 20n + 50 \in \mathcal{O}(n^2)$ avec $(73, 1)$, et la figure exhibe $(5, 13)$. Vérifiez que $(5, 13)$ convient en testant $n = 12$ puis $n = 13$:

$n = 12$: $f(12) =$ _____ $5 \times 12^2 =$ _____ inégalité vraie? _____
 $n = 13$: $f(13) =$ _____ $5 \times 13^2 =$ _____ inégalité vraie? _____

(c) Peut-on conclure de (b) que 13 est le plus petit seuil possible *quel que soit c* ? _____

9 Exercice 3 — Vrai ou faux

Cochez, sans justification.

Affirmation	Vrai	Faux
$n^2 + n \in \mathcal{O}(n^2)$		
$n^2 + n \in \mathcal{O}(n^3)$		
$n^2 + n \in \Theta(n^3)$		
$n \log n \in \Omega(n)$		
$3^n \in \mathcal{O}(2^n)$		
$\mathcal{O}$ est synonyme de « pire cas »		

Pour la dernière ligne, expliquez votre réponse en une phrase : _____

10 Exercice 4 — Analyse de cas

On considère la recherche linéaire d'une valeur dans un tableau de n éléments.

Cas	Entrée qui le réalise	Classe
Meilleur		
Pire		
Moyen		

(a) Pourquoi le pire cas sert-il de référence plutôt que le cas moyen ? Donnez deux raisons.

(b) Décrivez l'entrée réalisant le pire cas pour la recherche d'un doublon par double boucle avec sortie anticipée : _____

11 Exercice 5 — Récurrences

Résolvez par le théorème principal. Pour chacune, donnez a , b , $\alpha = \log_b a$, le cas applicable et la conclusion.

Réurrence	a	b	α	Cas et conclusion
$T(n) = T(n/2) + \Theta(1)$				
$T(n) = 2T(n/2) + \Theta(1)$				
$T(n) = 2T(n/2) + \Theta(n)$				
$T(n) = 3T(n/2) + \Theta(n^2)$				

(a) Les deux premières lignes ne diffèrent que par a . Expliquez, sans calcul, pourquoi l'écart de conclusion est si grand : _____

(b) À laquelle de ces récurrences correspond la recherche dichotomique ? _____

12 Exercice 6 — Question subsidiaire

Vos mesures de la Partie I ne donnent probablement pas un rapport exactement égal à 4 quand n double.

(a) Citez deux causes possibles, l'une liée à la machine, l'autre à la théorie elle-même : _____

(b) Un écart mesuré invalide-t-il la prédiction asymptotique ? Argumentez en deux phrases. _____

Pour aller plus loin (optionnel)

- Tracer les courbes mesurées avec matplotlib (le conteneur dispose de Python et matplotlib) et superposer $n^2/2$ ajusté par un facteur constant. La forme doit coïncider, pas les valeurs.
- Instrumenter votre `ArrayDeque` du Lab S01 : compter les déplacements réellement effectués par `agrandir()` sur une séquence de n insertions, et comparer à n .
- Reprendre la file naïve sur `ArrayList` avec `remove(0)` et mesurer n `dequeue` successifs.

Aide-mémoire

Question	Réponse
<code>s = s + i</code>	2 opérations
Comparaisons d'une boucle de n tours	$n + 1$
Boucles successives	on additionne, le maximum l'emporte
Boucles imbriquées	on multiplie
Pas multiplicatif (<code>i *= k</code>)	$\Theta(\log n)$ tours
Prouver $f \in \mathcal{O}(g)$	exhiber $c > 0$ et n_0
Rapport de temps quand n double	≈ 2 si linéaire, ≈ 4 si quadratique
$T(n) = aT(n/b) + f(n)$	comparer $f(n)$ à $n^{\log_b a}$
Compiler / tester / exécuter	<code>gradle build test run --no-daemon</code>

Rapport individuel — obligatoire

Chaque étudiant rédige un rapport de **1 à 2 pages** :

1. **Ce que j'ai fait** : fragments instrumentés, tests écrits, mesures obtenues.
2. **Les observations clés** : le tableau de mesures confirme-t-il la prédiction du CM ? Où l'écart apparaît-il ?
3. **Les difficultés** : formules de comptage fausses, mesures instables, blocages sur les exercices théoriques.
4. **Ce que j'ai appris** : le concept le plus important retenu de cette séance.

Format : PDF. **Nom** : `rapport-lab-S02-PRENOM-NOM.pdf`